

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 03 · LECTURE NOTES

Heuristic Search

A and where the information comes from.*

MODULE OUTCOMES

- State A* and prove its optimality conditions.
- Distinguish admissibility from consistency and know when each matters.
- Explain dominance and the effective branching factor.
- Derive heuristics from relaxations and pattern databases.
- Choose a memory-bounded or suboptimal variant deliberately.

1 A*

$$f(n) = g(n) + h(n)$$

Cost to reach n, plus the estimated cost from n to a goal — so $f(n)$ estimates the cost of the best complete solution *through* n, and expanding the lowest f means expanding in order of how promising the whole solution looks. **The two degenerate cases are the most useful way to see it: $h = 0$ gives uniform cost search** (optimal and slow, expanding everything within the optimal cost), and **$g = 0$ gives greedy best-first** (fast and not optimal, because it ignores what it has already spent). **A* is the combination, and it is the only point on that line with both properties** — which makes it inevitable rather than clever.

ADMISSIBLE FOR OPTIMALITY; CONSISTENT FOR EFFICIENCY

Admissible: $h(n)$ never overestimates the true remaining cost to the nearest goal. This is the condition that makes A* optimal, and the reason is worth holding: an overestimate can make the node on the optimal path look worse than a node that is not, so A* expands the wrong one and may reach a goal by a worse route before the better route's f value comes up.

Consistent (or monotone): $h(n) \leq \text{cost}(n, n') + h(n')$ for every edge n to n' . A triangle inequality on the heuristic — the estimate cannot drop by more than the step cost.

Consistency implies admissibility, and it is the condition that lets you **close a node permanently on first expansion**, because f is then non-decreasing along any path and the first expansion of a state is along an optimal path to it. **With an inconsistent (but admissible) heuristic you must allow reopening closed nodes**, or graph search loses optimality.

Nearly every natural heuristic is consistent, so this rarely bites in practice — **but it bites hard when it does**, and the symptom is suboptimal paths returned by an implementation using a heuristic you have correctly proved admissible. It is worth knowing the distinction purely so that this symptom is diagnosable, because otherwise it looks like a code bug and is not.

2 What a heuristic is worth

A BETTER HEURISTIC REDUCES THE EXPONENT'S BASE

If $h_1(n) \geq h_2(n)$ at every node and both are admissible, then h_1 *dominates* h_2 , and A* with h_1 never expands more nodes than A* with h_2 — a clean result with no caveats.

So always prefer the larger admissible heuristic. And the pointwise maximum of several admissible heuristics is itself admissible, which is a free improvement whenever you can afford to compute all of them — and the cost-benefit of doing so is exactly the trade in the table below.

The right measure is the effective branching factor b^* , defined by $N = (b^*)^d$ where N is the nodes expanded and d the solution depth — the uniform branching factor that would have produced the same node count. **Dropping b^* from 2.8 to 1.4 at depth 20 is a factor of about a million**, not a factor of two, because the improvement is in the base of an exponential.

Which is why heuristic design beats micro-optimisation by an enormous margin, and why §3's systematic derivation is one of the field's genuinely good results — it converts the largest available lever from a matter of inspiration into a procedure.

Heuristic	Cost to compute	Nodes expanded
$h = 0$	Free.	Everything within the optimal cost — the uniform cost baseline.
Manhattan distance (grid)	$O(1)$ per node.	Far fewer. The standard default for grid pathfinding, and nearly always the right starting point.

Heuristic	Cost to compute	Nodes expanded
Pattern database	Hours, once, offline; then $O(1)$ per lookup.	Orders of magnitude fewer — the strongest practical option for puzzle-like problems (§3).
Exact h	As hard as solving the original problem.	Exactly the solution path and nothing else — and therefore useless, since computing it was the problem.

The right point on that table depends on how many queries you will make. An offline precomputation costing hours and reused a million times is obviously correct; for a single query it is obviously not — and in a game, where the same level is searched thousands of times per session, the amortisation argument favours precomputation far more than it does in a one-shot academic benchmark. This is the argument behind Module 04 §3's precomputed hierarchies.

3 Where heuristics come from

1. RELAXATION -- remove a constraint from the problem and solve the relaxed version exactly. The relaxed solution cost is an ADMISSIBLE heuristic for the original, because every real solution is also a relaxed solution and therefore costs at least as much.

8-puzzle, allow tiles to pass through each other:
each tile moves independently

-> MANHATTAN DISTANCE summed over tiles.

Admissible, $O(1)$, and the standard choice.

8-puzzle, allow a tile to teleport anywhere:

-> MISPLACED TILE COUNT. Admissible, weaker, dominated by Manhattan.

THIS IS CSCE 669 MODULE 10's BOUND-FROM-A-RELAXATION ARGUMENT, in a different subject, with the inequality pointing the same way. Same idea, exactly.

2. PATTERN DATABASE -- abstract the state so that only some components are tracked, solve EVERY abstract state exhaustively offline by backward breadth-first search, and store the exact abstract costs. Look up at search time in $O(1)$.

Very strong. Memory-hungry. Precomputed once.

DISJOINT pattern databases can be ADDED rather than maxed, if the subproblems share no moves -- which multiplies rather than merely improves their strength.

3. LEARNED -- fit h from solved instances. Fast to evaluate, and ADMISSIBILITY IS LOST, so optimality goes with it. Sometimes an acceptable trade. Say so when you make it, and report the suboptimality you measured.

Relaxation is the one to internalise. It turns heuristic design from an act of invention into a mechanical procedure: list the problem's constraints, drop one, check whether the relaxed problem is easy, and if so you have an admissible heuristic with a one-line proof. **And it generalises — Module 07 §3 derives planning heuristics by exactly this method, automatically, from the action descriptions,** which is what made modern planners work.

4 When A* is too expensive

- **IDA*** — iterative deepening on the f value rather than on depth. **Linear memory, still optimal, and it repeats work** (Module 02 §2's constant-factor argument applies). **The right choice when memory is the binding constraint**, which Module 02 argued is usual.

- **Weighted A***: $f = g + w \cdot h$ with $w > 1$. Returns a solution at most w times the optimal cost, and does so far faster because the heuristic is effectively trusted more. **A bounded and stated compromise**, which is the kind worth making.
- **Beam search** — keep only the best k nodes at each level and discard the rest. Fast, constant memory, **no guarantee at all, and it can discard the only path to the goal** and report failure on a solvable problem.
- **Anytime A*** — return a solution quickly, then continue improving it while time remains, reporting the current best when interrupted. **The right shape for a game loop** (Module 04 §4), because the deadline is external and non-negotiable.
- **And bidirectional A***, which is substantially harder than bidirectional uniform-cost search because the two searches' f values are not comparable and the meeting condition is subtle — worth knowing exists, and not worth implementing casually.
- **Weighted A* is the one to reach for first. $w = 1.5$ is typically several times faster and within a few percent of optimal in practice**, well inside its 50% bound — and the bound is proved, so the compromise is quantified rather than hoped.

REPORT NODES EXPANDED, NOT SECONDS

Wall-clock time confounds the algorithm with your implementation, your language, your data structures, and the efficiency of your successor function — four things that have nothing to do with the question you are usually asking.

Nodes expanded isolates the algorithmic question, and it is the quantity the theory predicts, **so it is the number that can be compared against a textbook, a paper, or someone else's implementation**.

Report both, though. A heuristic that halves the nodes expanded and triples the time per node is a net loss, and only reporting both reveals it — which is a real and common outcome with expensive heuristics.

And report the effective branching factor (§2), which normalises for solution depth and is therefore **the only figure comparable across problem instances of different sizes** — nodes expanded alone conflates the heuristic's quality with the instance's difficulty.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapter 3 (sections on informed search)	A* , admissibility , consistency , and the relaxation argument of §3 , with the 8-puzzle worked through.
Hart, Nilsson & Raphael — A Formal Basis for the Heuristic Determination of Minimum Cost Paths (free)	The original A* paper , with the optimality proof. Short and readable.

Resource	What it covers for this module
Korf — Disjoint Pattern Database Heuristics (free)	The §3 pattern database method , including the additivity condition that makes them so strong.
Red Blob Games — Heuristics (free, interactive)	The practical guide for grid heuristics , including weighted A* and tie-breaking, with interactive demonstrations.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement A*** on top of Module 02's generic loop and verify it reduces to uniform cost at $h = 0$.
- 2 **Prove Manhattan distance admissible** for the 8-puzzle via the relaxation argument.
- 3 **Construct an inadmissible heuristic** and exhibit a problem where A* returns a suboptimal path.
- 4 **Construct an admissible but inconsistent heuristic** and show that closing nodes permanently loses optimality.
- 5 **Compare misplaced-tiles against Manhattan** and confirm dominance empirically.
- 6 **Compute the effective branching factor** for each and report the ratio.
- 7 **Build a pattern database** for a subset of the 15-puzzle tiles and measure the improvement.
- 8 **Implement weighted A*** and plot nodes expanded and solution cost against w from 1.0 to 3.0.
- 9 **Confirm the w -bound holds** on every instance.
- 10 **Implement IDA*** and compare peak memory against A*.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State A* and explain why the combination of g and h is necessary.
- 2 Define admissibility and consistency, and say what each buys.
- 3 What is the symptom of an inconsistent heuristic in graph search?
- 4 Define dominance, and state the free improvement it implies.
- 5 What is the effective branching factor and why is it the right measure?
- 6 Derive two 8-puzzle heuristics by relaxation.
- 7 What other course does the relaxation argument come from?
- 8 What is a pattern database, and when can two be added?

- 9 Name four variants of A* and the compromise each makes.
- 10 Why report nodes expanded rather than seconds?